Understanding Method 2: The Discrete Burgers Circuit

A Pedagogical, Illustrated Guide to Dislocation Detection in the MTM Code

MTM Lattice-Triangulation Documentation

October 2026

Abstract

Why does counting triangles around each atom (`NodeValence` and `ReferenceCoordination`) show 5/7 dislocations inside a **perfectly undeformed square crystal**? And how can we detect true dislocations, dipoles, and junctions reliably whether remeshing is turned on or off? This guide explains **Method 2 (the Discrete Burgers Circuit)** from scratch using plain physical intuition, worked numerical examples, and six visual diagrams.

Contents

1 The Puzzle: Why Does `NodeValence` Show Fake Dislocations in a Perfect Square Lattice? 2

1.1 Why Triangle Counting Works on a Triangular Lattice 2

1.2 Why Triangle Counting Fails on a Square Lattice 2

2 First Clarifying $\boldsymbol{F} = \boldsymbol{F}_e\boldsymbol{F}_p$: Elastic vs. Plastic Distortion 2

3 Method 2 Explained Step-by-Step: The Discrete Burgers Circuit 4

3.1 Step 1: Measure Each Directed Triangle Edge in Lattice Units 4

3.2 Step 2: Round Each Bond Vector to the Nearest Ideal Integer Step $\Delta\boldsymbol{n}_{ab}$ 4

3.3 Step 3: Sum the Three Integer Steps Around the Triangle 5

3.4 Worked Example 1: Why / and \ Diagonals ALWAYS Give $\boldsymbol{B}_T = (0, 0)$ 5

3.5 Worked Example 2: What Happens at a Real Dislocation Core? 6

4 How Bigger Loops Work Automatically: Cancellation of Interior Edges 6

5 Quick Guide: Which Fields to Select in ParaView 8

1 The Puzzle: Why Does `NodeValence` Show Fake Dislocations in a Perfect Square Lattice?

In ParaView, the simplest way to look for crystal defects in a 2D triangulated mesh is to count how many triangles touch each atom. In our code, this number is called `NodeValence` (or `ReferenceCoordination`).

1.1 Why Triangle Counting Works on a Triangular Lattice

In a 2D **triangular (hexagonal) lattice**, every atom is surrounded by 6 nearest neighbors at the exact same distance a_0. All triangles in the mesh are equilateral triangles with side lengths (a_0, a_0, a_0). Therefore:

- Every atom in a perfect triangular crystal is touched by **6 triangles**.
- If an atom is touched by **5** or **7** triangles, there is genuinely an extra or missing half-plane of atoms (a true dislocation core).

1.2 Why Triangle Counting Fails on a Square Lattice

Now consider a 2D **square lattice** with grid spacing $a_0 = 1$. Each atom has only **4 nearest neighbors** (up, down, left, right at distance 1). To build a triangular finite-element mesh, every 1×1 square cell must be cut in half by a **diagonal edge** of length $\sqrt{2} \approx 1.414$.

Here is the catch: every square has **two equally valid diagonals** of the exact same length $\sqrt{2}$:

- the **forward slash** diagonal (/), or
- the **backslash** diagonal (\).

Because both diagonals have the exact same length $\sqrt{2}$, tiny floating-point rounding (10^{-15}) during Delaunay triangulation or remeshing will flip some squares from / to \ **even when every single atom sits at its exact perfect square lattice position!**

Look at Figure 1 to see what happens to the number of triangles touching each atom when just **one single square** in a perfect grid has a flipped diagonal:

In general, any interior atom i in a square lattice is surrounded by 4 squares. It always touches the 4 horizontal/vertical grid edges, plus anywhere from $d_i \in \{0, 1, 2, 3, 4\}$ diagonals:

$$\text{Triangle Valence at Atom } i \;=\; 4 + d_i \;\in\; \{4, 5, 6, 7, 8\}. \tag{1}$$

Whenever a simulation creates regions with / diagonals next to regions with \ diagonals, the boundary between those regions lights up with fake 5s and 7s!

Why We Need a Geometry-Aware Method.

Counting triangles only looks at **graph connectivity** (how many mesh lines meet at a point) and ignores **where the atoms actually are in space**. To know if a dislocation is truly present, we must measure the **Burgers vector** around a closed loop of atoms.

2 First Clarifying $\boldsymbol{F} = \boldsymbol{F}_e\boldsymbol{F}_p$: Elastic vs. Plastic Distortion

Before introducing Method 2, let us clarify the fundamental continuum formula

$$\boldsymbol{F} = \boldsymbol{F}_e\boldsymbol{F}_p \tag{2}$$

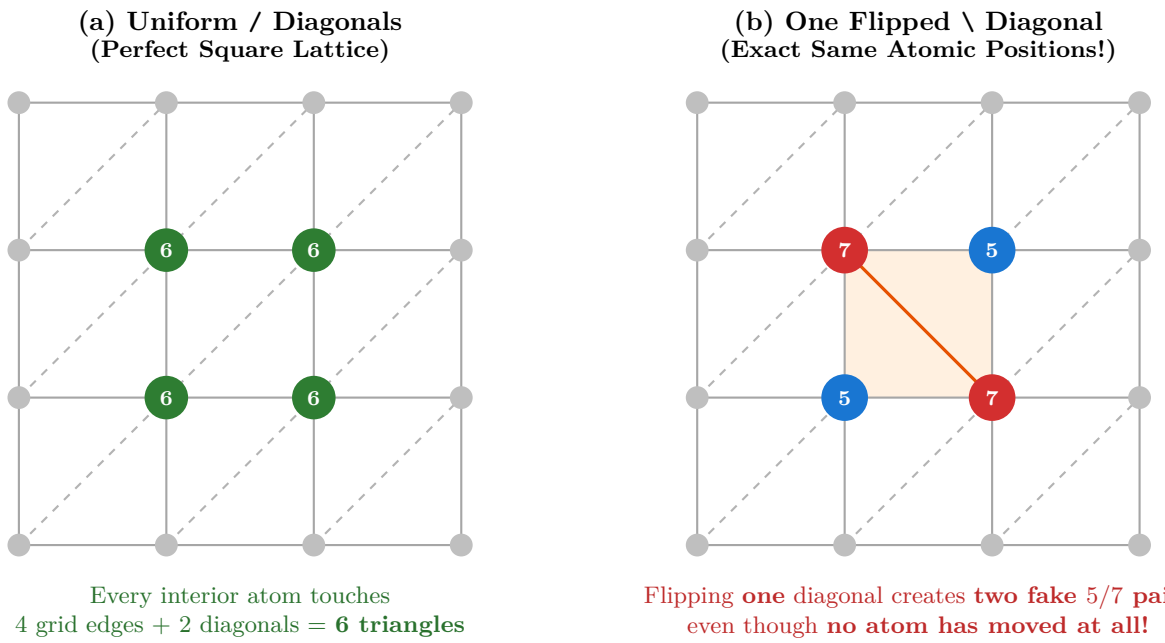


Figure 1: **Why `NodeValence` fails on a square lattice.** In both (a) and (b), every atom sits at its exact undeformed square grid position $(m, n) \in \mathbb{Z}^2$. Simply flipping the middle diagonal from / to \ changes the triangle count at the four corners from $(6, 6, 6, 6)$ to $(5, 7, 5, 7)$.

and why remeshing changes how we use it.

Suppose we take a single square cell of atoms and shear the top row to the right by **one full lattice spacing** a_0 (Figure 2):

1. **Total deformation gradient $\boldsymbol{F}$:** Compared to who each atom was originally connected to at $t = 0$, the cell has been sheared by $\gamma = 1$:
$$\boldsymbol{F} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

2. **Plastic slip $\boldsymbol{F}_p$:** Because shifting a crystal by exactly 1 full lattice spacing brings the atoms right back onto the ideal square grid, this shear costs **zero energy**. It is a pure lattice-invariant plastic slip:
$$\boldsymbol{F}_p = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \in \mathrm{SL}(2, \mathbb{Z}), \qquad \boldsymbol{M} = \boldsymbol{F}_p^{-1} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix},$$
where $\boldsymbol{M}$ is the integer matrix found by **Lagrange reduction** in our code!

3. **Elastic distortion $\boldsymbol{F}_e$:** Once we strip away the integer plastic slip $\boldsymbol{F}_p$, what remains is the true elastic stretch/rotation of the crystal bonds:
$$\boldsymbol{F}_e = \boldsymbol{F}\boldsymbol{F}_p^{-1} = \boldsymbol{F}\boldsymbol{M} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \boldsymbol{I}.$$
Because $\boldsymbol{F}_e = \boldsymbol{I}$, the crystal is stress-free!

Now look at Panel (c) of Figure 2: **when remeshing is turned ON**, the code reconnects atom B directly to atom D (which is now right above B) and resets the reference triangle! So on the new mesh, $\boldsymbol{F}_{\text{new}} = \boldsymbol{I}$ and $\boldsymbol{M}_{\text{new}} = \boldsymbol{I}$. That is why we cannot simply look for jumps in $\boldsymbol{M}$ when remeshing is enabled—remeshing has already absorbed $\boldsymbol{M}$ into the new mesh connections!

Instead, we can measure the elastic lattice step **directly from the bond vectors of the current mesh**. That is **Method 2**.

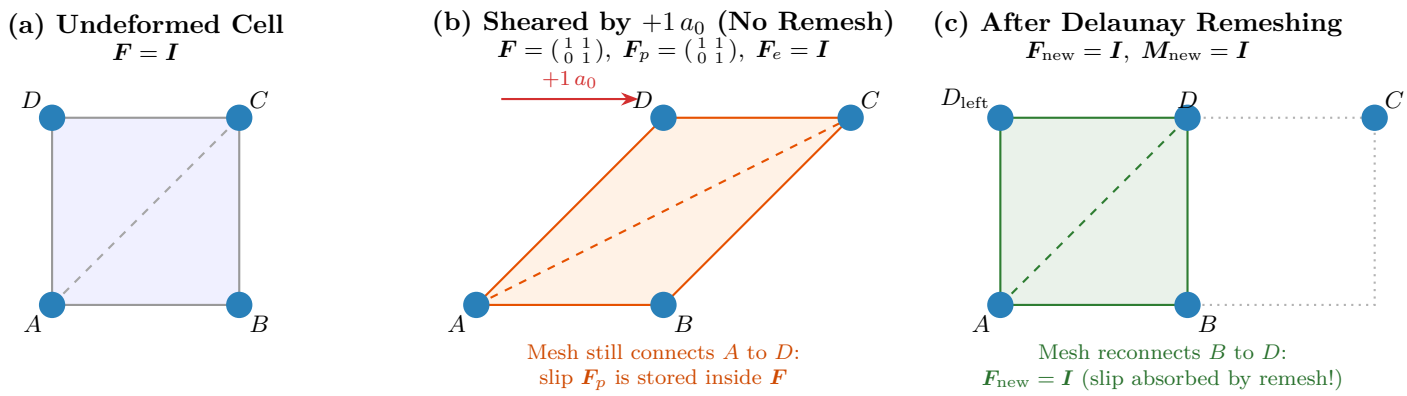


Figure 2: **How $\boldsymbol{F} = \boldsymbol{F}_e\boldsymbol{F}_p$ works before and after remeshing.** Without remeshing (b), the triangle remembers its original partners, so the plastic slip $\boldsymbol{F}_p$ is visible inside $\boldsymbol{F}$. Once Delaunay remeshing reconnects atoms to their new neighbors (c), the new triangle has $\boldsymbol{F}_{\text{new}} = \boldsymbol{I}$.

3 Method 2 Explained Step-by-Step: The Discrete Burgers Circuit

In textbook physics, how do you find out if a region of crystal contains a dislocation? You walk around a closed loop of atoms (a **Burgers circuit**), counting how many ideal lattice steps (*right, up, left, down*) you take from atom to atom.

- If you end up back at $(0, 0)$ in the ideal grid, **there is no dislocation inside the loop**.
- If your step count fails to close by $\boldsymbol{B} \neq (0, 0)$, **there is a dislocation with Burgers vector $\boldsymbol{B}$ inside the loop!**

Method 2 applies this exact textbook idea to the **smallest possible closed loop in our mesh: every single triangle T!**

3.1 Step 1: Measure Each Directed Triangle Edge in Lattice Units

Take any triangle $T = (A, B, C)$ oriented counter-clockwise. It has three directed edges: $A \to B$, $B \to C$, and $C \to A$. For an edge from atom a at position $\boldsymbol{x}_a$ to atom b at position $\boldsymbol{x}_b$:

$$\text{Continuous bond vector in lattice units:} \qquad \boldsymbol{s}_{ab} = \boldsymbol{A}_0^{-1}(\boldsymbol{x}_b - \boldsymbol{x}_a), \tag{3}$$

where $\boldsymbol{A}_0$ is the ideal lattice basis matrix:

- For an unrotated square lattice with spacing $a_0 = 1$, $\boldsymbol{A}_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, so $\boldsymbol{s}_{ab} = \boldsymbol{x}_b - \boldsymbol{x}_a$ is simply $(\Delta x, \Delta y)$ in units of a_0!
- For a triangular lattice, $\boldsymbol{A}_0 = a_0 \begin{pmatrix} 1 & 1/2 \\ 0 & \sqrt{3}/2 \end{pmatrix}$ converts $(\Delta x, \Delta y)$ into steps along the $0°$ and $60°$ crystal axes.

3.2 Step 2: Round Each Bond Vector to the Nearest Ideal Integer Step $\Delta\boldsymbol{n}_{ab}$

Even if the crystal is elastically stretched or sheared by 10% or 20%, a bond that is supposed to be $(1, 0)$ might have $\boldsymbol{s}_{ab} = (1.08, -0.05)$. When we round each component to the nearest integer, we recover the **ideal integer lattice step**:

$$\Delta\boldsymbol{n}_{ab} = \text{round}(\boldsymbol{s}_{ab}) = \begin{pmatrix} \text{round}(s_{ab,x}) \\ \text{round}(s_{ab,y}) \end{pmatrix} \in \mathbb{Z}^2. \tag{4}$$

For example, $\text{round}(1.08, -0.05) = (1, 0)$.

3.3 Step 3: Sum the Three Integer Steps Around the Triangle

Now walk counter-clockwise around the three edges of triangle $T = (A \to B \to C \to A)$ and add the three integer step vectors together:

$$\boxed{\boldsymbol{B}_T \;=\; \Delta\boldsymbol{n}_{AB} + \Delta\boldsymbol{n}_{BC} + \Delta\boldsymbol{n}_{CA} \;\in\; \mathbb{Z}^2.} \tag{5}$$

This integer vector $\boldsymbol{B}_T = (B_{T,x}, B_{T,y})$ is the **Discrete Burgers Vector of triangle** T.

3.4 Worked Example 1: Why / and \ Diagonals ALWAYS Give $\boldsymbol{B}_T = (0, 0)$

Let us test Eq. (5) on a square lattice with / vs. \ diagonals (Figure 3):

- **Case (a) — Triangle with a / diagonal:** Vertices at $A = (0, 0)$, $B = (1, 0)$, $C = (1, 1)$.

$$\begin{aligned} A \to B: &\quad \Delta\boldsymbol{n}_{AB} = (1,0) - (0,0) = (+1,\ 0) \\ B \to C: &\quad \Delta\boldsymbol{n}_{BC} = (1,1) - (1,0) = (\ 0, +1) \\ C \to A: &\quad \Delta\boldsymbol{n}_{CA} = (0,0) - (1,1) = (-1, -1) \quad \text{(the / diagonal!)} \end{aligned}$$

  Add them up:

$$\boldsymbol{B}_T = (+1, 0) + (0, +1) + (-1, -1) = \mathbf{(0, 0)}.$$

- **Case (b) — Triangle with a flipped \ diagonal:** Vertices at $A = (0, 0)$, $B = (1, 0)$, $D = (0, 1)$.

$$\begin{aligned} A \to B: &\quad \Delta\boldsymbol{n}_{AB} = (1,0) - (0,0) = (+1,\ 0) \\ B \to D: &\quad \Delta\boldsymbol{n}_{BD} = (0,1) - (1,0) = (-1, +1) \quad \text{(the \ diagonal!)} \\ D \to A: &\quad \Delta\boldsymbol{n}_{DA} = (0,0) - (0,1) = (\ 0, -1) \end{aligned}$$

  Add them up:

$$\boldsymbol{B}_T = (+1, 0) + (-1, +1) + (0, -1) = \mathbf{(0, 0)}.$$

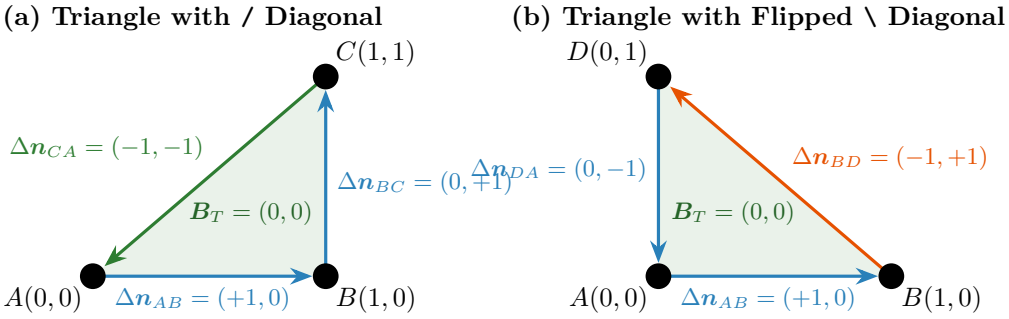


Figure 3: **Why diagonal flips never create false dislocations in Method 2.** Because a / diagonal is recognized as the integer vector $(+1, +1)$ and a \ diagonal is recognized as $(-1, +1)$, the vector sum of the three edges around the triangle is $\boldsymbol{B}_T = (0, 0)$ in **both** cases!

Look at Figure 3: because Method 2 knows that a diagonal edge has integer step $(\pm 1, \pm 1)$ instead of treating it like a $(1, 0)$ nearest-neighbor bond, **every triangle in a perfect or elastically deformed square lattice has $\boldsymbol{B}_T = (0, 0)$, regardless of how its diagonals are connected!**

3.5 Worked Example 2: What Happens at a Real Dislocation Core?

Now let us see what happens when there **is** a real edge dislocation inside the crystal! Suppose the bottom row has an extra atom squeezed in (an extra vertical half-plane of atoms, i.e. an edge dislocation with Burgers vector $\boldsymbol{B} = (+1, 0)$).

Look at the core triangle $T^* = (A \to B \to C \to A)$ right at the tip of the extra half-plane in Figure 4:

1. Along the bottom edge $A \to B$: atom B is one grid spacing to the right of A (slightly compressed to $\Delta x = 0.85$), which rounds to
$$\Delta \boldsymbol{n}_{AB} = \text{round}(0.85,\ 0.0) = (+1,\ 0).$$
2. Along the right edge $B \to C$: atom C on the upper row sits at $\Delta x = -0.42$ to the left of B and $\Delta y = +1.0$ up. Because $|-0.42| < 0.5$, it rounds to a pure vertical step:
$$\Delta \boldsymbol{n}_{BC} = \text{round}(-0.42,\ +1.0) = (0,\ +1).$$
3. Along the left edge $C \to A$: going from C back down to A has $\Delta x = -0.43$ and $\Delta y = -1.0$. Because $|-0.43| < 0.5$, it *also* rounds to a pure vertical step:
$$\Delta \boldsymbol{n}_{CA} = \text{round}(-0.43,\ -1.0) = (0,\ -1).$$

Now add the three integer steps around the core triangle T^*:

$$\boldsymbol{B}_{T^*} = \Delta \boldsymbol{n}_{AB} + \Delta \boldsymbol{n}_{BC} + \Delta \boldsymbol{n}_{CA} = (+1, 0) + (0, +1) + (0, -1) = \mathbf{(+1, 0)} \neq (0, 0)! \tag{6}$$

Bingo! The core triangle T^* immediately lights up with $\boldsymbol{B}_{T^*} = (+1, 0)$, pinpointing the exact location and the exact Burgers vector of the dislocation!

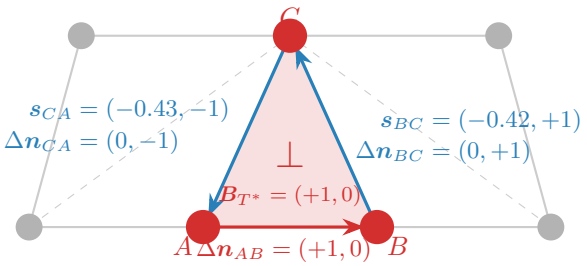


Figure 4: **How Method 2 detects a true edge dislocation core.** At the tip of an extra half-plane of atoms between A and B, the bottom bond $A \to B$ takes $+1$ horizontal lattice step, whereas both upward/downward bonds $B \to C$ and $C \to A$ have horizontal components $< 0.5\, a_0$ and round to 0 horizontal steps. Their sum around T^* is $\boldsymbol{B}_{T^*} = (+1, 0) \neq (0, 0)$.

4 How Bigger Loops Work Automatically: Cancellation of Interior Edges

What if we want the Burgers circuit around a **whole group of triangles**—for example, the **1-ring of triangles surrounding an atom** i, or a region containing a **dislocation junction**?

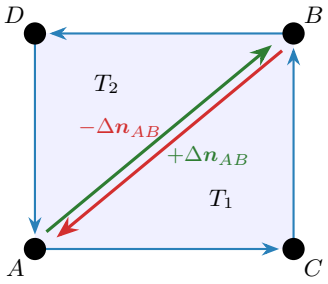
Here is the most beautiful property of Method 2: **you never have to trace outer loops by hand! You just add $\boldsymbol{B}_T$ of the triangles inside the region.**

Why? Look at Figure 5(a), where two adjacent triangles T_1 and T_2 share an interior edge between atoms A and B:

- When we walk counter-clockwise around T_1, we cross the shared edge from $A \to B$, adding $+\Delta \boldsymbol{n}_{AB}$.
- When we walk counter-clockwise around T_2, we cross the **exact same edge in the opposite direction** $B \to A$, adding $\Delta \boldsymbol{n}_{BA} = -\Delta \boldsymbol{n}_{AB}$.

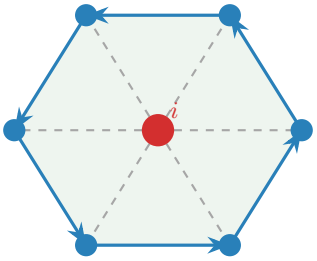
When we add $\boldsymbol{B}_{T_1} + \boldsymbol{B}_{T_2}$, the shared interior edge $+\Delta \boldsymbol{n}_{AB} - \Delta \boldsymbol{n}_{AB} = (0, 0)$ **cancels out in exact integer arithmetic!**

(a) Two Triangles Sharing an Edge
$\boldsymbol{B}_{T_1} + \boldsymbol{B}_{T_2}$ = Outer 4-Atom Loop



Shared edge traversed once as $A \to B$ and once as $B \to A$: **cancels to 0!**

(b) Nodal 1-Ring Circuit Around Atom i
$\boldsymbol{B}_i^{(1\text{-ring})} = \sum_{T \ni i} \boldsymbol{B}_T$



All dashed interior spokes cancel; only the outer blue polygon remains!

Figure 5: **Discrete Stokes' Theorem in action.** Because every shared interior edge is crossed in opposite directions by its two adjacent triangles, summing $\boldsymbol{B}_T$ over any patch of triangles automatically equals the closed Burgers circuit around the outer boundary of that patch.

This gives us three powerful results immediately:

1. **Unit-Cell Square Plaquette Filter (Figure 5a):** In a square lattice, the diagonal $A \leftrightarrow B$ has length $\sqrt{2}\, a_0 \approx 1.414\, a_0$, which is longer than the four outer sides of the square $(1.0\, a_0)$. If strong elastic shear causes *only* that internal diagonal to round by ± 1 while the four outer edges of the square are still perfect, then T_1 gets $+\boldsymbol{B}$ and T_2 gets $-\boldsymbol{B}$, so their sum on the 4-atom square is $\boldsymbol{B}_{T_1} + \boldsymbol{B}_{T_2} = (0, 0)$. The code automatically detects when two triangles sharing a diagonal edge ($L > 1.15\, a_0$) have $\boldsymbol{B}_{T_1} + \boldsymbol{B}_{T_2} = (0, 0)$ and cancels that pair to zero!

2. **Nodal 1-Ring Burgers Vector (Figure 5b):** For every atom i, summing $\boldsymbol{B}_T$ over all triangles touching atom i,
$$\boldsymbol{B}_i^{(1\text{-ring})} = \sum_{T \ni i} \boldsymbol{B}_T, \tag{7}$$
gives the exact Burgers circuit around the 1-ring polygon of neighbors of atom i!
   - If $\boldsymbol{B}_i^{(1\text{-ring})} = (0, 0)$, atom i is in a defect-free crystal environment, so we set `ReferenceCoordination` $= 6$.
   - If $\boldsymbol{B}_i^{(1\text{-ring})} \neq (0, 0)$, atom i sits directly on a true dislocation core, and we preserve its 5/7 defect label!

3. **Dislocation Junctions (Frank's Rule, Figure 6):** What if a horizontal edge dislocation $\boldsymbol{B}_1 = (+1, 0)$ (gliding in from the left boundary in `Shift / Staircase`) collides with a vertical edge dislocation $\boldsymbol{B}_2 = (0, +1)$ (gliding up from the bottom boundary) to form a **dislocation junction**? By Discrete Stokes' Theorem, any circuit enclosing the junction automatically sums the Burgers vectors of the reacting dislocations:
$$\boldsymbol{B}_{\text{junction}} = \sum_{T \subset \Omega_{\text{junction}}} \boldsymbol{B}_T = \boldsymbol{B}_1 + \boldsymbol{B}_2 = (+1, 0) + (0, +1) = (+1, +1), \tag{8}$$
with magnitude $\|\boldsymbol{b}_{\text{junction}}\|_2 = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414\, a_0$!

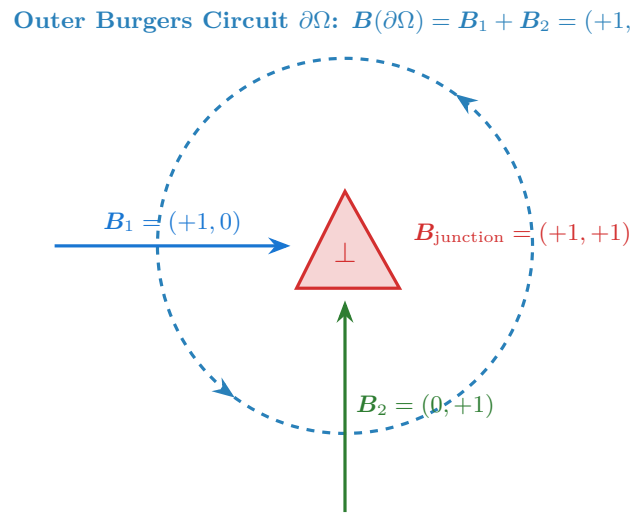


Figure 6: **Why Method 2 works automatically at dislocation junctions.** When a horizontal dislocation $\boldsymbol{B}_1 = (+1, 0)$ and a vertical dislocation $\boldsymbol{B}_2 = (0, +1)$ meet at a junction, summing $\boldsymbol{B}_T$ over the junction triangles automatically obeys Frank's rule $\boldsymbol{B}_{\text{junction}} = \boldsymbol{B}_1 + \boldsymbol{B}_2 = (+1, +1)$.

5 Quick Guide: Which Fields to Select in ParaView

When you open `vtk_output/configuration_XXXXX.vtk` or `vtk_defects/defects_XXXXX.vtk` in ParaView, you now have the following fields available in the coloring dropdown:

ParaView Field Name	Data Type	What It Shows & When to Use It
`ReferenceCoordination`	Point Scalar	**Best drop-in replacement for old coordination plots:** equals `6` everywhere in a defect-free crystal (zero fake 5/7s!) and shows `5` or `7` *only* at true dislocation cores where $\boldsymbol{B}_i^{(\text{1-ring})} \neq \boldsymbol{0}$.
`BurgersMagnitude`	Point Scalar	Magnitude $\|\boldsymbol{b}_i^{(\text{1-ring})}\|_2$ around each atom (0 in perfect crystal, 1 at an elementary dislocation core, $\sqrt{2}$ at a $(1,1)$ junction).
`BurgersVector`	Point Vector	2D vector $(b_{i,x}, b_{i,y}, 0)$ at each atom—use with ParaView's **Glyph** filter (Arrows) to see the exact slip direction of each dislocation!
`ElementBurgersMagnitude`	Cell Scalar	Magnitude $\|\boldsymbol{b}_T\|_2$ on each individual triangle T—pinpoints the exact core triangle containing the dislocation.
`ElementBurgersVector`	Cell Vector	2D vector $(b_{T,x}, b_{T,y}, 0)$ on each triangle T.
`NodeValence`	Point Scalar	Raw unfiltered triangle count (kept only so you can still inspect the raw Delaunay mesh connectivity if desired).